

Writing Functions in Java

Practice set 1 — answer key and marking notes | for the instructor

How to use this key

For every question the key gives the expected function signature, a complete working version, the reasoning behind each type choice, and a short list of things to check while marking. Students may pick different names and a slightly different body; that is fine. What matters is the shape of the function: the number and types of the inputs, and the type of the output. All ten answers below were compiled and run together as one Java program.

Q	Takes in	Gives back	Pattern being tested
1	int	boolean	Single input, yes/no output
2	int, double	double	Two inputs of different types
3	double	char	Number in, single character out
4	String	boolean	Text in, decision out
5	double, boolean	double	Yes/no as an input
6	String, String, int, double	void	Many inputs, no output
7	int[]	int[] (or a record)	List in, two values out
8	Citizen (object)	String	Object in
9	String, String, String	Member (object)	Object out, state kept between calls
10	nothing	double	No input

Question 1: Can this person vote?

One piece of information in, a yes/no decision out

Expected signature

```
static boolean isEligibleToVote(int age)
```

Complete answer

```
static boolean isEligibleToVote(int age) {  
    return age >= 18;  
}
```

Why these types

- Age in completed years is a whole number, so the input is **int**.
- A yes/no answer is a **boolean**. Returning the comparison directly is cleaner than an if/else that returns true or false.

Sample call

```
System.out.println(isEligibleToVote(17)); // false  
System.out.println(isEligibleToVote(18)); // true
```

What to check when marking

- ✓ Input type is int (not double, not String).
- ✓ Return type is boolean (not int 0/1, not a printed "Yes").
- ✓ Boundary: 18 must give true.

Question 2: Monthly electricity bill

Two pieces of information in, one amount out

Expected signature

```
static double calculateBill(int unitsUsed, double ratePerUnit)
```

Complete answer

```
static double calculateBill(int unitsUsed, double ratePerUnit) {  
    return unitsUsed * ratePerUnit;  
}
```

Why these types

- Units are counted in whole numbers: **int**. The rate has paise, so it is a **double**.
- Multiplying an int by a double gives a double, so the amount must come back as **double**.
- Two inputs of different types show that each input gets its own type in the declaration.

Sample call

```
System.out.println(calculateBill(240, 6.75)); // 1620.0
```

What to check when marking

- ✓ Two inputs, in a sensible order, each with its own type.
- ✓ Return type double (an int return would silently drop the paise).

Question 3: Grade letter for a student

One number in, one letter out

Expected signature

```
static char gradeFor(double percentage)
```

Complete answer

```
static char gradeFor(double percentage) {  
    if (percentage >= 90) {  
        return 'A';  
    } else if (percentage >= 75) {  
        return 'B';  
    } else if (percentage >= 60) {  
        return 'C';  
    } else if (percentage >= 40) {  
        return 'D';  
    } else {  
        return 'F';  
    }  
}
```

Why these types

- A percentage like 78.5 needs decimals: **double**.
- A single letter is a **char**, written with single quotes ('A'). A String "A" would also work but char is the more precise type for exactly one letter.
- Checking from the highest band downward means each condition only needs a lower limit.

Sample call

```
System.out.println(gradeFor(78.5)); // B
```

What to check when marking

- ✓ Return type is char (accept String if the student explains why).
- ✓ Boundaries: 90 gives A, 75 gives B, 40 gives D, 39.9 gives F.
- ✓ Every path returns a value (no missing else).

Question 4: Is this a valid Indian mobile number?

Text in, a yes/no decision out

Expected signature

```
static boolean isValidMobile(String number)
```

Complete answer

```
static boolean isValidMobile(String number) {  
    if (number == null || number.length() != 10) {  
        return false;  
    }  
    for (int i = 0; i < number.length(); i++) {  
        if (!Character.isDigit(number.charAt(i))) {  
            return false;  
        }  
    }  
    char first = number.charAt(0);  
    return first >= '6' && first <= '9';  
}
```

Why these types

- The number is handled as typed text, so the input is a **String**, not a number. A leading zero, a space, or 10 digits that overflow an int are all reasons not to use a numeric type.
- The result is yes/no: **boolean**.
- Returning false as soon as a rule fails keeps the code readable.

Sample call

```
System.out.println(isValidMobile("9845012345")); // true  
System.out.println(isValidMobile("98450 12345")); // false
```

What to check when marking

- ✓ Input is String, not int or long.
- ✓ All three rules are checked: length, all digits, first digit 6-9.
- ✓ Return type boolean.

Question 5: City bus fare

A number and a yes/no in, an amount out

Expected signature

```
static double busFare(double distanceKm, boolean isSeniorCitizen)
```

Complete answer

```
static double busFare(double distanceKm, boolean isSeniorCitizen) {  
    double fare = 10 + 2 * distanceKm;  
    if (isSeniorCitizen) {  
        fare = fare / 2;  
    }  
    return fare;  
}
```

Why these types

- Distance may be 7.5 km: **double**. The senior-citizen flag is a yes/no, so it is a **boolean** input, which is the first time a boolean appears as an input rather than an output.
- Fare can have paise after the discount: return **double**.

Sample call

```
System.out.println(busFare(7.5, true)); // 12.5
```

What to check when marking

- ✓ Second input is boolean (not int 0/1, not String "yes").
- ✓ Discount applied to the whole fare including the Rs 10 boarding charge.

Question 6: Print a shop receipt line

Several pieces of information in, nothing given back (only printing)

Expected signature

```
static void printReceiptLine(String customerName, String itemName,
                             int quantity, double unitPrice)
```

Complete answer

```
static void printReceiptLine(String customerName, String itemName,
                             int quantity, double unitPrice) {
    double lineTotal = quantity * unitPrice;
    System.out.printf("%s | %s | %d x Rs %.2f = Rs %.2f\n",
                      customerName, itemName, quantity, unitPrice, lineTotal);
}
```

Why these types

- Four inputs of three different types: two **Strings**, one **int**, one **double**.
- Nothing is handed back, so the return type is **void** and there is no return statement with a value.
- printf with %.2f keeps two decimal places; String.format works the same way.

Sample call

```
printReceiptLine("Ravi", "Sugar 1kg", 2, 48.00);
// Ravi | Sugar 1kg | 2 x Rs 48.00 = Rs 96.00
```

What to check when marking

- ✓ Return type is void; the function prints rather than returns.
- ✓ Quantity is int, unit price is double.
- ✓ Output matches the given format exactly (spaces, |, x, =, two decimals).

Question 7: Busiest and quietest day at the public library

A list of numbers in, two numbers out

Expected signature

```
static int[] highestAndLowest(int[] dailyCounts)
```

Complete answer

```
static int[] highestAndLowest(int[] dailyCounts) {
    int highest = dailyCounts[0];
    int lowest = dailyCounts[0];
    for (int count : dailyCounts) {
        if (count > highest) {
            highest = count;
        }
        if (count < lowest) {
            lowest = count;
        }
    }
    return new int[] { highest, lowest };
}
```

Why these types

- A list of whole numbers is an **int[]** (array of int).
- Java functions return exactly one thing. To hand back two numbers, pack them into one container: here an **int[]** of length 2, where position 0 is the highest and position 1 is the lowest.
- An alternative, and better for readability, is a small class or record: **record MinMax(int highest, int lowest) {}** with return type **MinMax**. Either answer is acceptable.
- Starting both trackers from the first element (not from 0) makes the function correct even when all counts are the same or all negative.

Sample call

```
int[] result = highestAndLowest(new int[] {120, 95, 210, 180, 60});
System.out.println(result[0] + " " + result[1]); // 210 60
```

What to check when marking

- ✓ Input is **int[]** (or **List<Integer>**).
- ✓ Two values are returned together via an array, a record, or a small class, not printed.
- ✓ Trackers start from the first element, not from 0.

Question 8: Address label for a citizen

One bundle of related information in, one line of text out

Expected signature

```
static String addressLabel(Citizen c)    // with a Citizen class
```

Complete answer

```
class Citizen {
    String name;
    String houseNumber;
    String street;
    int wardNumber;
    int pinCode;

    Citizen(String name, String houseNumber, String street,
            int wardNumber, int pinCode) {
        this.name = name;
        this.houseNumber = houseNumber;
        this.street = street;
        this.wardNumber = wardNumber;
        this.pinCode = pinCode;
    }
}

static String addressLabel(Citizen c) {
    return c.name + ", #" + c.houseNumber + ", " + c.street
        + ", Ward " + c.wardNumber + ", Bengaluru - " + c.pinCode;
}
```

Why these types

- When several related pieces of information always travel together, they are grouped into a class. The input is then a single **Citizen** object instead of five separate inputs.
- House number is a **String** because "42A" is not a number. Ward and PIN are **int**.
- The label is one line of text: return type **String**.

Sample call

```
Citizen c = new Citizen("Lakshmi Rao", "42", "5th Cross", 31, 560035);
System.out.println(addressLabel(c));
```

What to check when marking

- ✓ A class (or record) is declared with the five fields and correct types.
- ✓ The function takes one Citizen input, not five loose inputs.
- ✓ Return type String; the label is returned, not printed.

Question 9: Register a new customer

Three pieces of information in, one complete record out

Expected signature

```
static Member registerMember(String name, String email, String mobile)
// together with a Member class
```

Complete answer

```
class Member {
    int membershipNumber;
    String name;
    String email;
    String mobile;

    Member(int membershipNumber, String name, String email, String mobile) {
        this.membershipNumber = membershipNumber;
        this.name = name;
        this.email = email;
        this.mobile = mobile;
    }
}

static int nextMembershipNumber = 1001;

static Member registerMember(String name, String email, String mobile) {
    Member m = new Member(nextMembershipNumber, name, email, mobile);
    nextMembershipNumber = nextMembershipNumber + 1;
    return m;
}
```

Why these types

- Three **String** inputs; the mobile number stays text for the same reason as in Question 4.
- The result is a whole record, so the return type is the class itself: **Member**. This is the first question where an object is the output.
- The running membership number must survive between calls, so it lives outside the function as a static field. A local variable inside the function would reset to 1001 every time.

Sample call

```
Member a = registerMember("Anjali", "anjali@mail.com", "9900011122");
Member k = registerMember("Kiran", "kiran@mail.com", "9900033344");
System.out.println(a.membershipNumber + " " + k.membershipNumber); // 1001 1002
```

What to check when marking

- ✓ A Member class with four fields of correct types.
- ✓ Return type is Member (an object), not String or void.
- ✓ The counter is kept outside the function and increases on every call.

Question 10: Standard water tariff

Nothing in, one fixed value out

Expected signature

```
static double standardWaterTariff()
```

Complete answer

```
static double standardWaterTariff() {  
    return 7.00;  
}
```

Why these types

- Empty brackets mean the function takes nothing in. It still needs the brackets to be a function.
- The rate has decimals: return **double**.
- Keeping the value in one function means a change to the tariff is made in exactly one place. A **static final double** constant is an equally good answer.

Sample call

```
double bill = 12.5 * standardWaterTariff(); // 87.5
```

What to check when marking

- ✓ Empty parameter list.
- ✓ Return type double with the value returned, not printed.